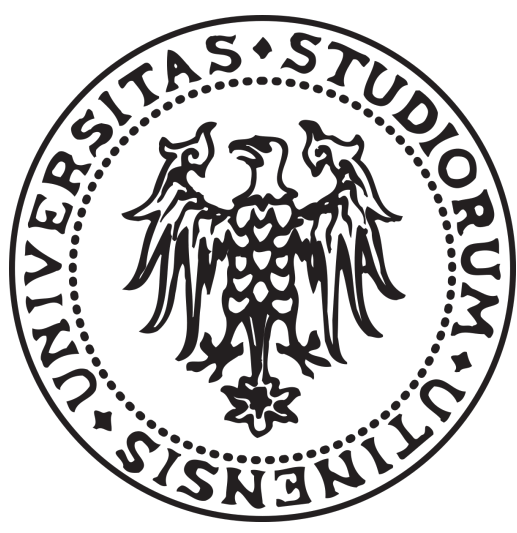




INGEGNERIA DEL SOFTWARE - 2025-26

PROGETTAZIONE ARCHITETTURALE DEL SOFTWARE

LEZIONE 21
10/12/2025
VINCENZO RICCIO

RIFERIMENTI

- Sommerville - Capitolo 6

Vincenzo Riccio
Ingegneria del Software 2025/2026
Università degli Studi di Udine



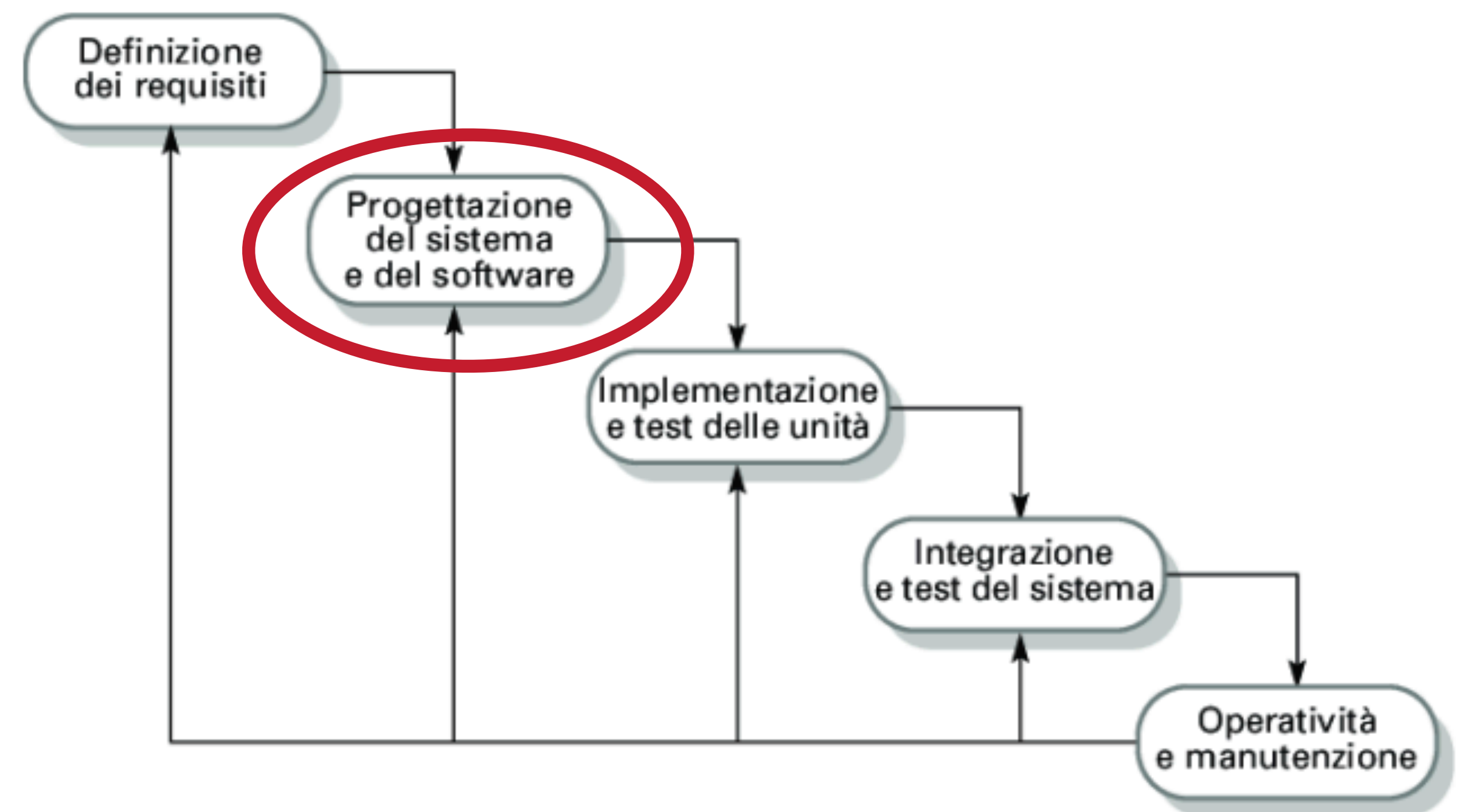


PROGETTAZIONE DEL SOFTWARE

ATTIVITÀ DI PROGETTAZIONE

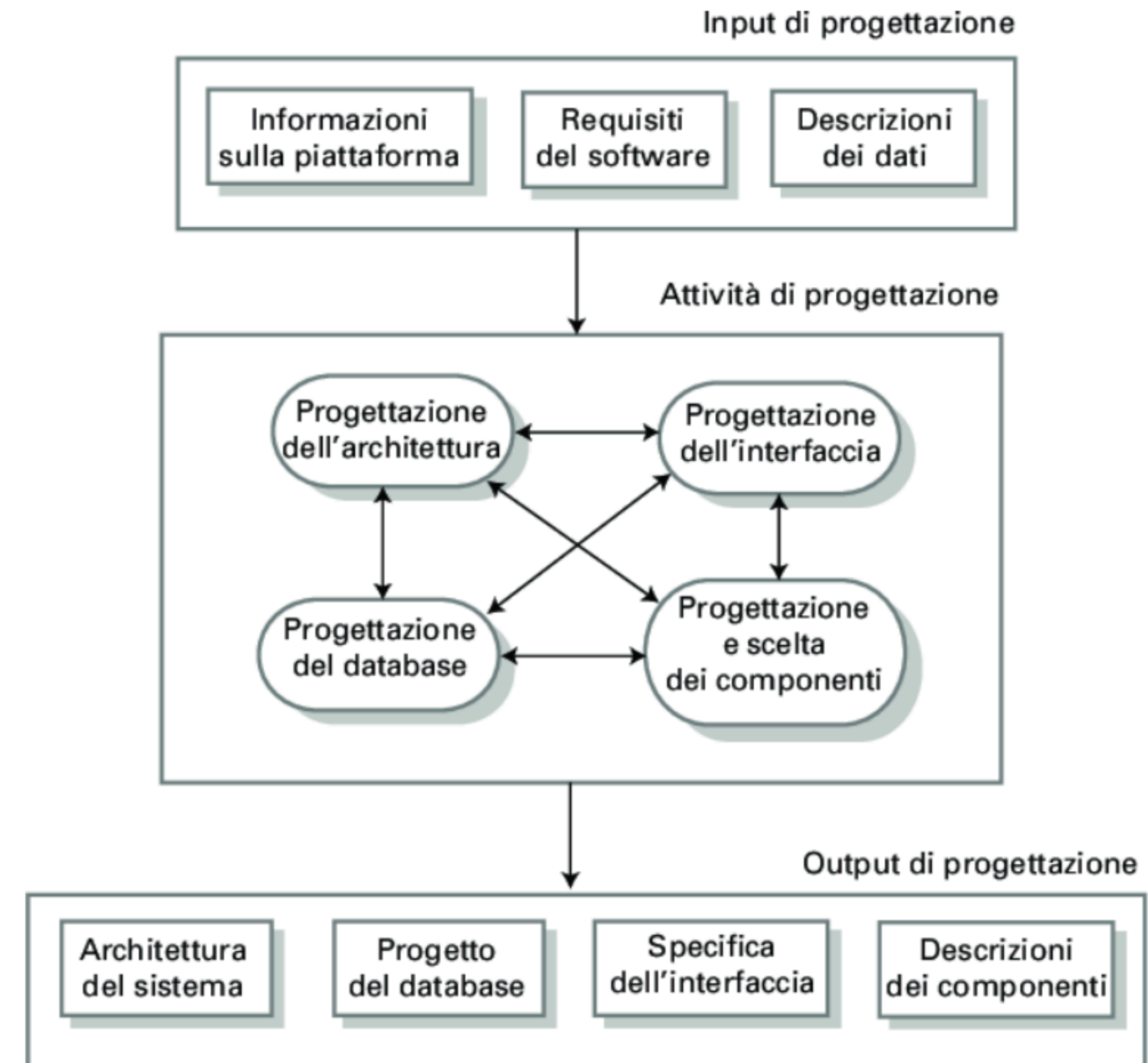


- **Ruolo chiave nello sviluppo:** La progettazione architeturale si colloca nella fase di Progettazione del sistema e rappresenta il collegamento critico tra l'ingegneria dei requisiti e la progettazione di dettaglio di ciascun componente



PROGETTAZIONE: FINALITÀ

- Progettare significa eseguire un processo di risoluzione di un problema il cui obiettivo è trovare e descrivere un modo:
 - per implementare i requisiti funzionali
 - rispettando i vincoli imposti dai requisiti non funzionali (inclusi i vincoli sul budget)
 - in conformità con una serie di principi di buona qualità

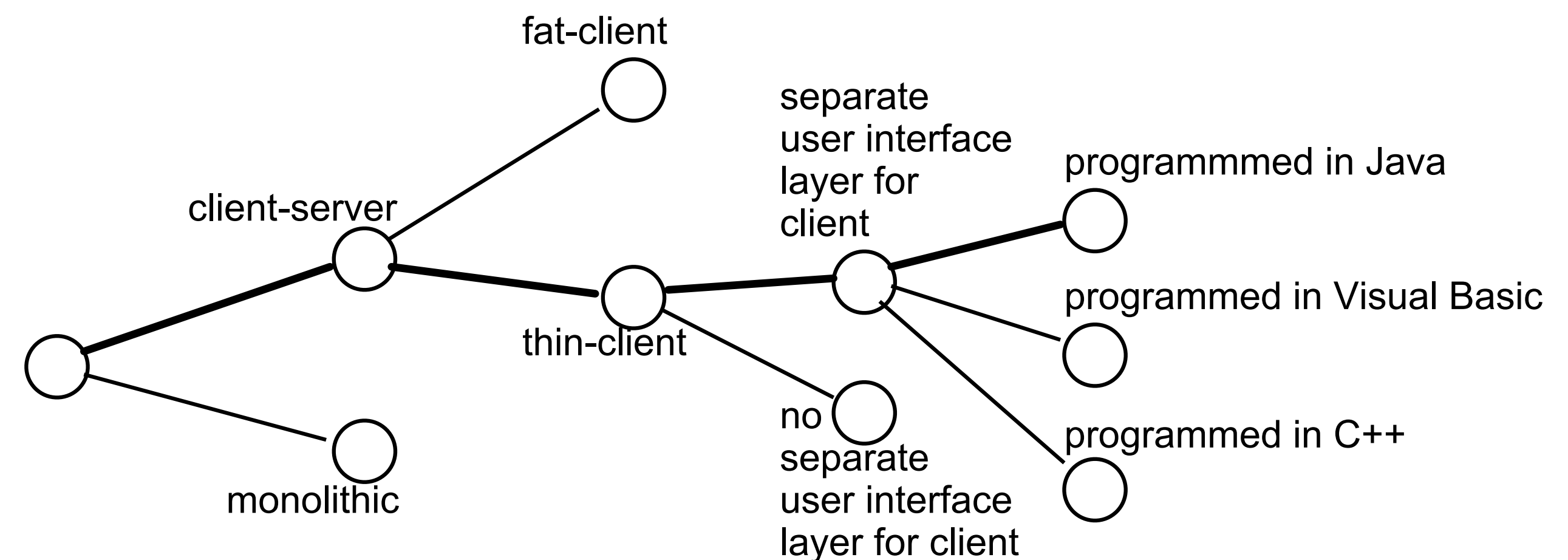


DECISIONI DI PROGETTAZIONE



- Un/a progettista affronta una serie di problemi di progettazione (*design*)
- Ciascun problema ha normalmente una serie di soluzioni alternative, le cosiddette **design options**
- Il/la progettista effettua una serie di **design decision** per risolvere ciascun problema scegliendo la *migliore* fra le alternative possibili in base ai requisiti specifici del sistema ma anche del background e dell'esperienza degli architetti

- **Design space:** insieme dei possibili progetti che saranno implementabili scegliendo tra le varie alternative



STRATEGIE DI PROGETTAZIONE: TOP-DOWN



- Parte col progettare la struttura di altissimo livello del sistema
 - Es. definire l'architettura del software e il tipo di database che sarà impiegato
- Quindi procede gradualmente con decisioni più dettagliate riguardanti aspetti di più basso livello, quali:
 - Il formato di particolari dati
 - Gli algoritmi che saranno usati
 - Le interfacce utente

STRATEGIE DI PROGETTAZIONE: BOTTOM-UP



- Parte progettando i componenti basilari di basso livello e si decide poi come collegarli insieme per ottenere i componenti di più alto livello.
- Si procede dunque per livelli di astrazione (*layers of abstraction*) sempre più alti



- Il processo di progettazione si traduce quindi in una serie di decisioni da prendere
- Non esiste un processo di progettazione unico, esso dipende dal sistema stesso ma anche dalle conoscenze ed esperienze del team di ingegneri
- In genere è normale usare un misto dei due approcci Top-Down e Bottom-Up
 - ◎ La progettazione Top-Down è necessaria per definire una buona struttura del sistema.
 - ◎ La progettazione Bottom-Up serve a progettare componenti riusabili in altri punti del sistema

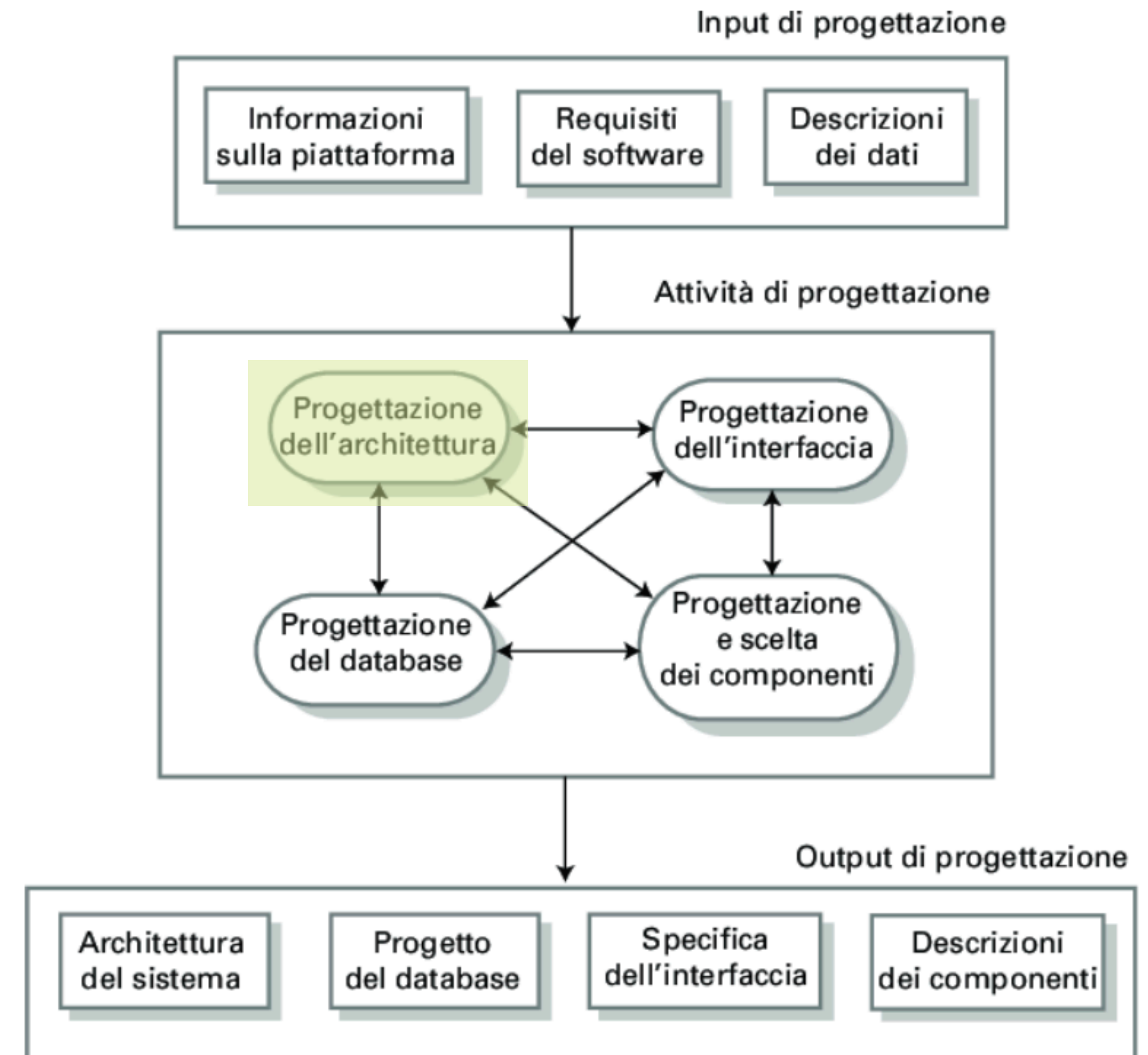


PROGETTAZIONE ARCHITETTURALE

PROGETTAZIONE ARCHITETTURALE



- Si occupa di comprendere come organizzare il sistema software e della progettazione della sua struttura complessiva
- Identifica i principali componenti strutturali di un sistema e le loro relazioni e interfacce
- L'output di tale processo è un modello architetturale che descrive come il sistema è organizzato e come i suoi componenti comunicano





"L'architettura software è l'organizzazione di base di un sistema, espressa dai suoi componenti, dalle relazioni tra di loro e con l'ambiente, e i principi che ne guidano il progetto e l'evoluzione."

[IEEE/ANSI 1471-2000]

- L'architettura del software è importante perché influisce (o addirittura pone vincoli) su prestazioni, robustezza, distribuzione e manutenibilità di un sistema
- I singoli componenti implementano i requisiti funzionali di un sistema, ma è l'architettura che ha un'influenza predominante sulle caratteristiche non funzionali di un sistema, così come i requisiti non funzionali hanno un effetto rilevante sull'architettura



- **Performance:** Preferire componenti grandi, localizzando le operazioni critiche e minimizzare le comunicazioni
- **Protezione:** Utilizzare un'architettura a strati, posizionando le risorse critiche negli strati interni, con convalida ad ogni strato
- **Sicurezza Funzionale (Safety):** Concentrare le funzionalità critiche per la sicurezza in un numero limitato di sottosistemi per fornire meccanismi di protezione e ridurre i costi
- **Disponibilità:** Componenti ridondanti e meccanismi per la tolleranza ai guasti
- **Manutenibilità:** Utilizzare componenti piccoli facilmente sostituibili/modificabili



- L'attività di progettazione architettuale può scontrarsi con i seguenti principi agili:
 - ◎ **Documentazione minima:** la necessità di produrre un progetto architettuale è generalmente accettata nei metodi agili nelle fasi iniziali del processo di sviluppo
 - ◎ **Refactoring:** é possibile rifattorizzare i singoli componenti ma rifattorizzare l'intera architettura è costoso poiché ciascun cambiamento all'architettura può interessare numerosi componenti



- La progettazione e documentazione esplicita dell'architettura ha diversi vantaggi:
 - Fornisce una presentazione del sistema ad alto livello che consente il confronto tra stakeholder
 - Durante la progettazione è analizzata la conformità del sistema ai requisiti non funzionali
 - Un modello architetturale può essere riusato per sistemi con requisiti simili (riuso su vasta scala)



PATTERN ARCHITETTURALI



- Un'architettura può essere riusabile per un insieme di sistemi
- I sistemi nello stesso dominio applicativo hanno spesso architetture simili che ne riflettono i concetti fondamentali
- Quando si progetta l'architettura di un nuovo sistema bisogna analizzare cosa ha in comune con le classi di applicazioni più generali e stabilire quanto di esse può essere riutilizzato
- Ad esempio le linee di prodotti (product lines) sono costruite attorno a un'architettura di base con varianti che specificano specifiche richieste dei clienti



"A pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice".

[C. Alexander et al., "A Pattern Language". Oxford University Press, New York, 1977]

- I pattern sono soluzioni a problemi di progettazione comuni e ricorrenti che si incontrano in applicazioni reali
- Un pattern architetturale é una descrizione dell'organizzazione del sistema che è stata utilizzata in diversi sistemi software



▸ **Riuso**

- ◎ Riuso della conoscenza/esperienza di progettazione
- ◎ Raramente i problemi sono nuovi e unici

▸ **Presentazione e comunicazione**

- ◎ Stabiliscono una terminologia comune e condivisa che facilita la comunicazione. Ad esempio è facile dire "Qui ci serve un'architettura client-server"
- ◎ Forniscono una prospettiva di alto livello del sistema
- ◎ Permettono la comprensione e la discussione del progetto architettuale senza dover gestire troppo presto i dettagli della progettazione

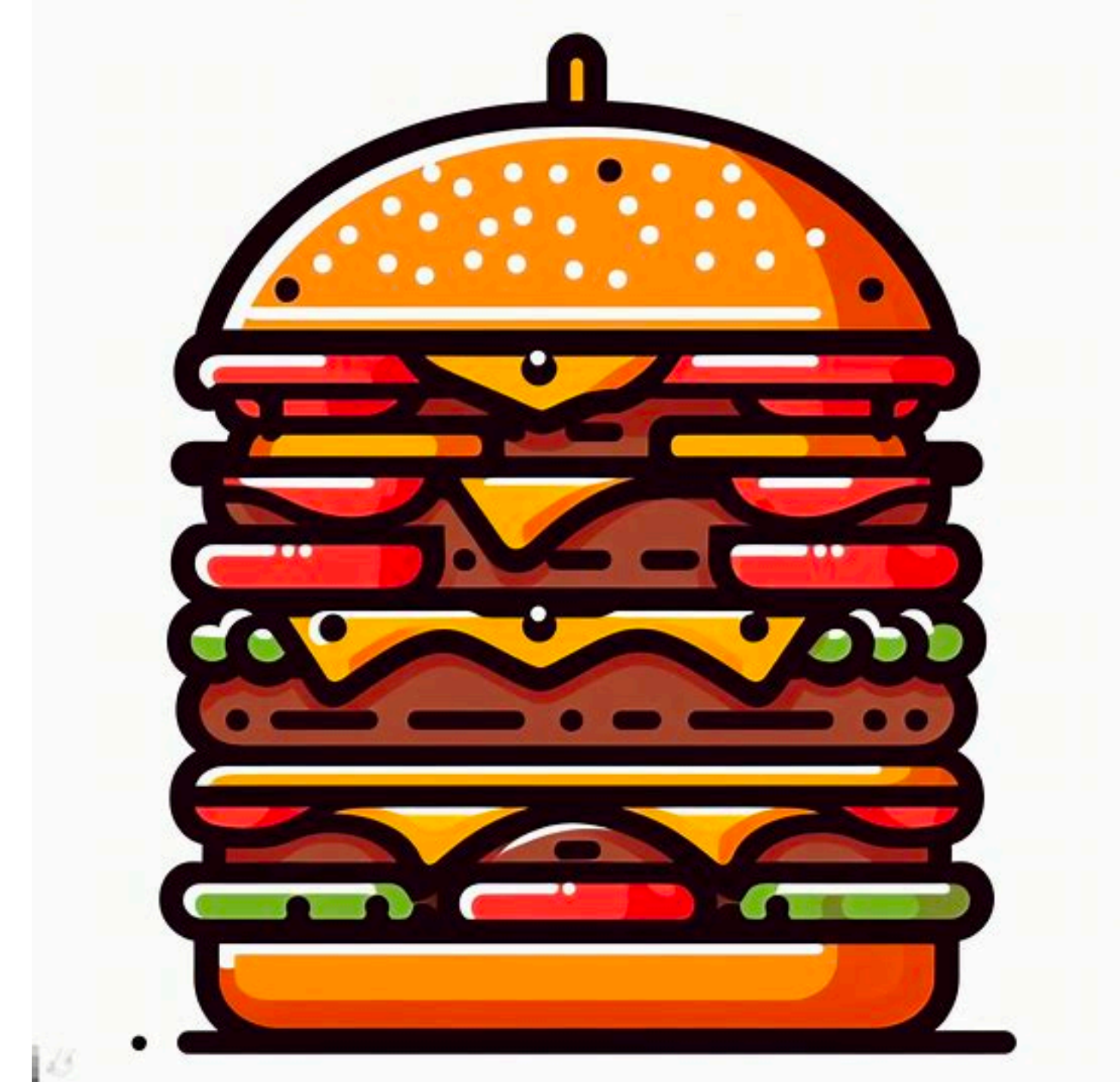


- Quando si prendono decisioni sull'architettura di un sistema, è importante conoscere gli schemi architeturali più comuni, i loro punti di forza e debolezza
- Un pattern descrive una **buona pratica**, che è stata già usata con successo e verificata in vari sistemi e ambienti
- Dovrebbe includere le informazioni su quando il suo utilizzo è appropriato e i dettagli sui suoi vantaggi e svantaggi
- I pattern possono essere descritti in modo standard usando template testuali e/o diagrammi (ad esempio in UML)

PATTERN ARCHITETTURALE MULTI-LAYER



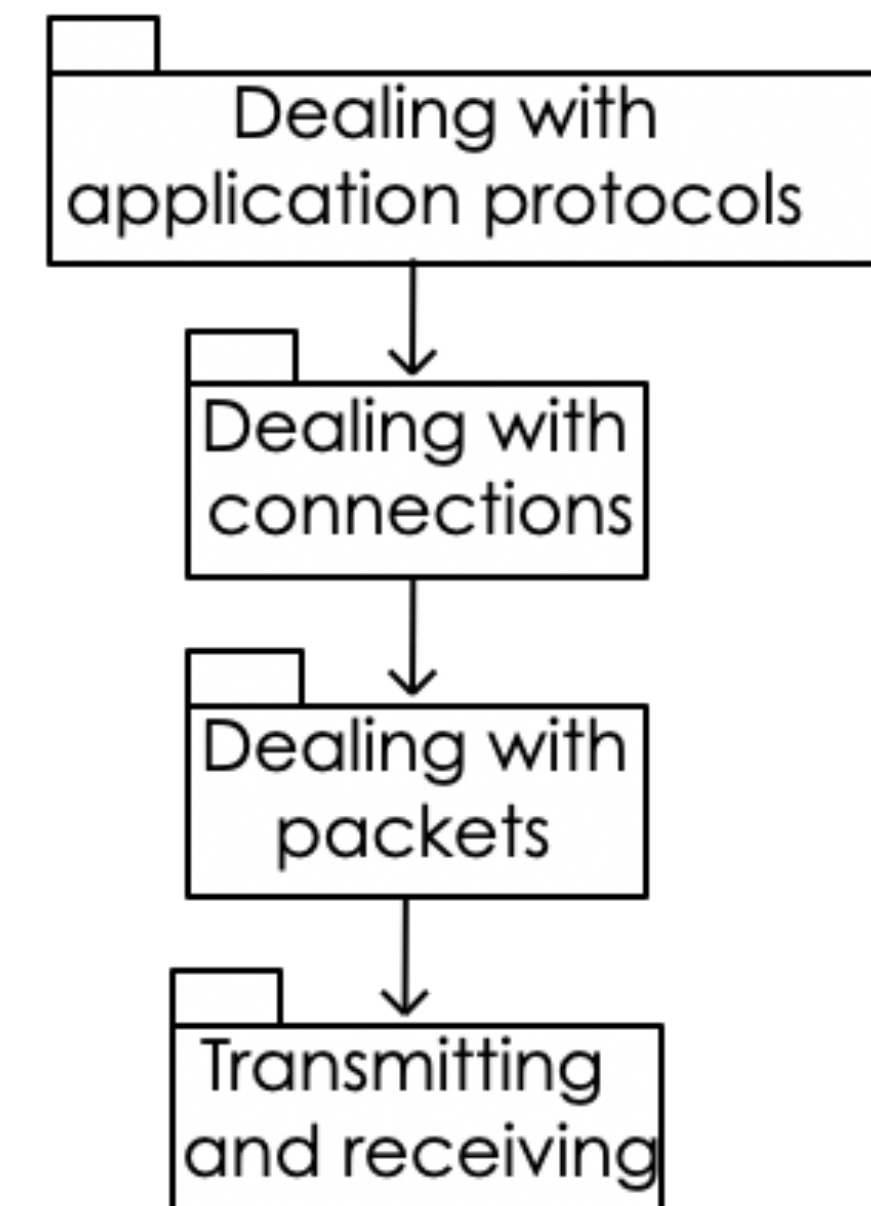
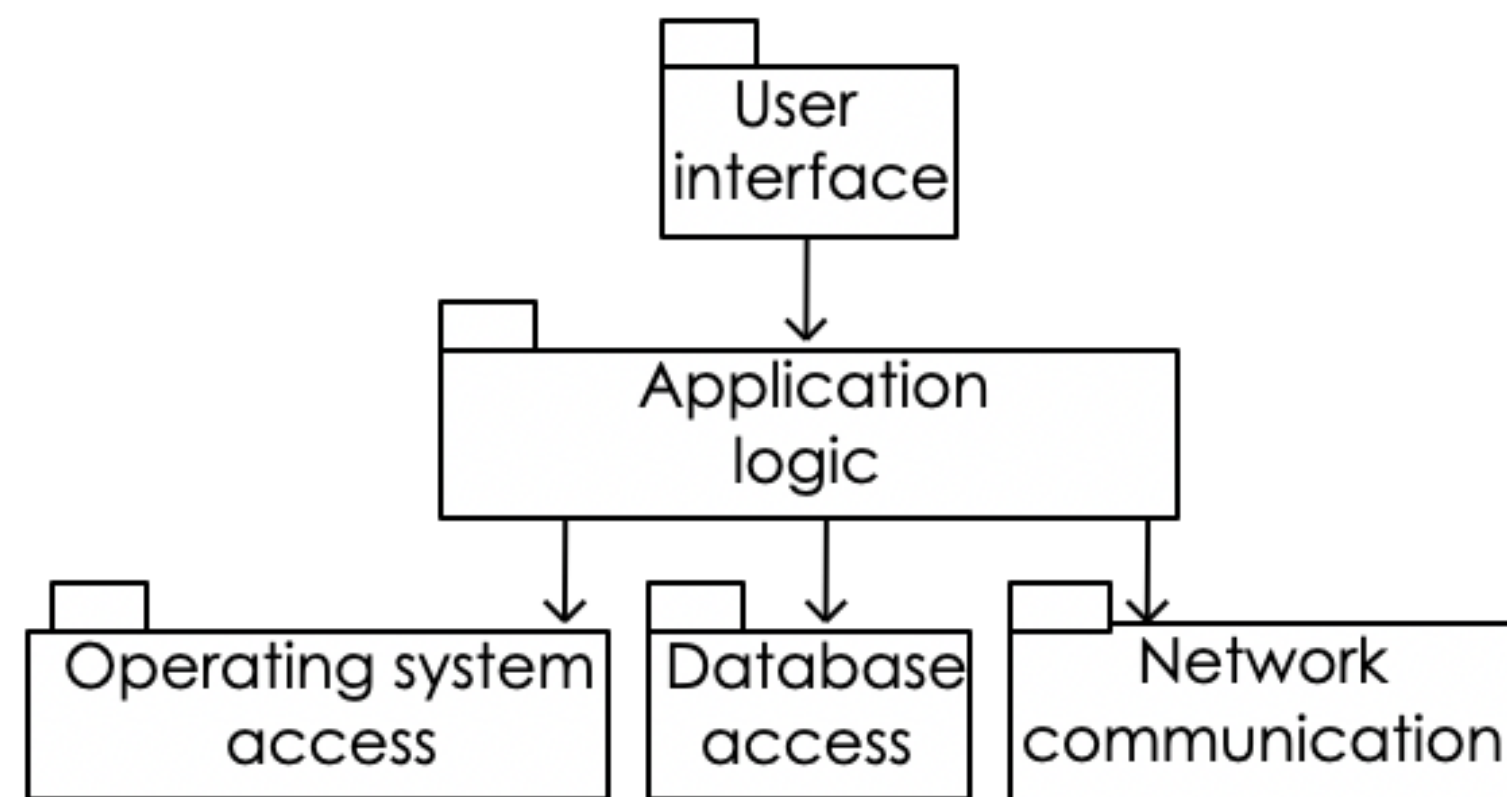
- Una decomposizione gerarchica di un sistema in un insieme ordinato di layer (strati)
- Un layer è un raggruppamento di sottosistemi che forniscono servizi correlati, eventualmente realizzati utilizzando servizi di altri layer
- Un sistema complesso può essere costruito stratificando strati aventi via via crescenti livelli di astrazione



PATTERN ARCHITETTURALE MULTI-LAYER



- In un'architettura multi-layer, ciascun livello comunica solo con il livello sottostante e non ha conoscenza dei layer dei livelli più alti
- Il livello più alto vede quello più in basso come un insieme di servizi
- Ogni livello offre un'interfaccia ben definita al livello immediatamente sovrastante





- **Separazione:** solitamente gli elementi che forniscono servizi correlati sono raggruppati nello stesso layer e sono separati dagli altri elementi
- **Indipendenza:** é possibile progettare e modificare uno strato indipendentemente dagli altri, purché vengano rispettate le interfacce
- Separazione ed indipendenza permettono di localizzare le modifiche: se si modifica l'interfaccia o si aggiungono funzionalità a uno strato, solo gli strati adiacenti ne sono influenzati
- Ideale per sistemi multipiattaforma: le dipendenze dalla macchina sono localizzate in strati specifici. Solo tali strati devono essere reimplementati per diverse piattaforme (ad es. sistema operativo)

PATTERN ARCHITETTURALE MULTI-LAYER: TEMPLATE

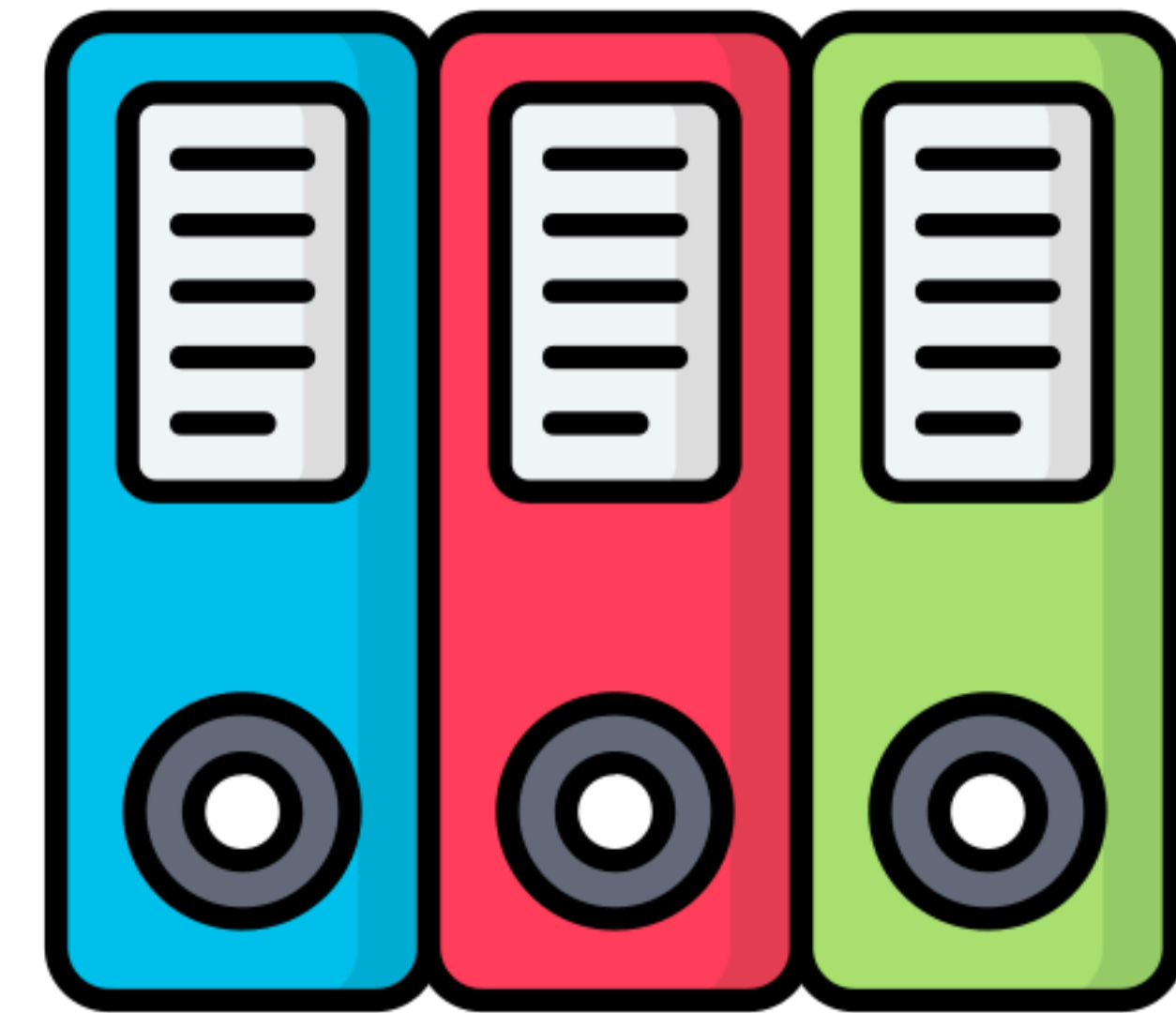


Nome	Architettura a strati
Descrizione	Organizza il sistema in vari strati, con funzionalità associate a ciascuno strato. Uno strato fornisce i servizi allo strato sopra di esso, quindi gli strati di livello più basso rappresentano i servizi di base che probabilmente saranno utilizzati in tutto il sistema.
Esempio	Un modello a strati di un sistema di apprendimento digitale che supporta l'apprendimento di tutti i soggetti nelle scuole
Quando si usa	Si usa per costruire nuove funzioni per un sistema esistente; quando lo sviluppo è distribuito fra più team, dove ogni team ha la responsabilità di sviluppare le funzioni di uno strato; quando c'è una richiesta di protezione su più livelli.
Vantaggi	Consente la sostituzione di interi strati, se l'interfaccia non viene modificata. Le funzioni ridondanti (per esempio, l'autenticazione) possono essere fornite in ciascuno strato per aumentare la fidatezza del sistema.
Svantaggi	Nella pratica spesso è difficile ottenere una netta separazione fra gli strati, e uno strato di alto livello potrebbe aver bisogno di interagire direttamente con strati di livelli inferiori, anziché con lo strato immediatamente sotto di esso. Le performance possono essere un problema, a causa dei vari livelli di interpretazione di una richiesta di servizio, essendo questa elaborata in ciascuno strato.

PATTERN ARCHITETTURA REPOSITORY



- Modello utilizzato quando ci sono diversi sotto-sistemi che devono condividere grandi quantità di dati comuni da memorizzare per lungo tempo
- I dati condivisi sono salvati e gestiti da un database centrale a cui tutti i sotto-sistemi possono accedere, detto **repository**
- Tutti i sotto-sistemi devono operare con un modello di dati concordato



PATTERN ARCHITETTURALE REPOSITORY



- Il repository è accessibile a tutti i componenti del sistema
- I componenti non si scambiano dati direttamente ma solo attraverso il repository





► **Vantaggi**

- ◎ I componenti possono essere indipendenti (un componente non deve per forza sapere dell'esistenza di un altro componente) e non devono trasmettere dati direttamente tra loro
- ◎ La repository gestisce tutti i dati in modo coerente (ad es. controllo accessi e backup)
- ◎ Semplice integrare nuovi componenti che si conformano al modello dei dati condiviso

► **Svantaggi**

- ◎ Difficile e meno efficiente distribuire il repository su più macchine (problemi di ridondanza e consistenza dei dati per mantenere coerenti e aggiornate più copie degli stessi dati)
- ◎ Il repository costituisce un single point of failure, tutti i suoi problemi influiscono sull'intero sistema
- ◎ Difficile integrare componenti non conformi al modello dei dati comune

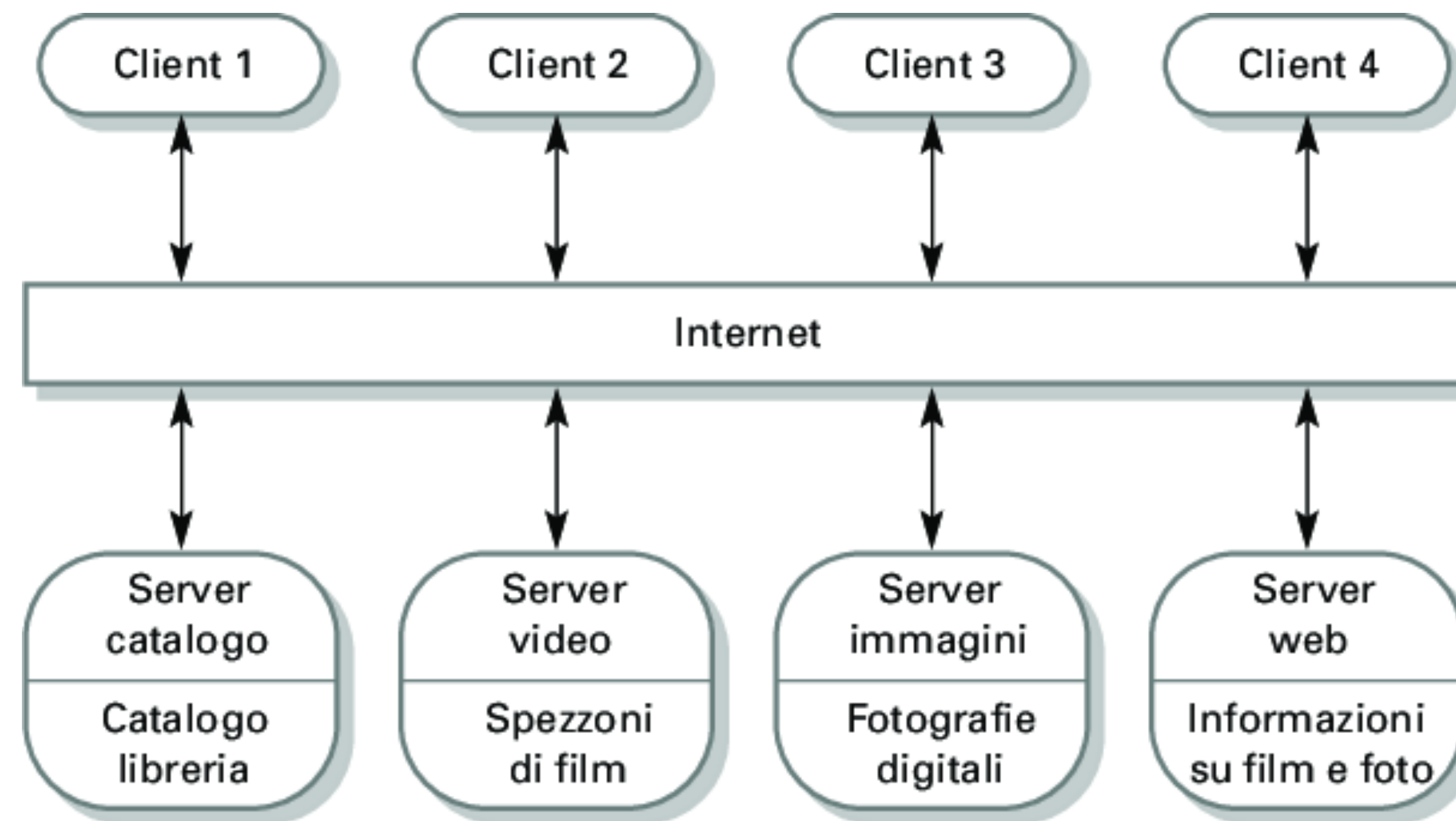
PATTERN ARCHITETTURALE CLIENT-SERVER



- Architettura **distribuita** in cui ciascun servizio è fornito da un server separato
- I client accedono ai server per utilizzare i servizi

I CLIENT SONO
SOLITAMENTE ESEGUITI
CONTEMPORANEAMENTE
SU COMPUTER
DIFFERENTI

I SERVER SONO
LOGICAMENTE SEPARATI
MA POSSONO ANCHE
ESSERE ESEGUITI SULLO
STESSO COMPUTER





► **Vantaggi**

- ◎ I server possono essere distribuiti su una rete
- ◎ Separazione e indipendenza: i servizi e i server sono indipendenti e possono essere modificati senza influire sulle altre parti del sistema
- ◎ Facile aggiungere un nuovo server ed integrarlo con il resto del sistema

► **Svantaggi**

- ◎ Ciascun servizio potrebbe essere un single point of failure, tuttavia esso potrebbe essere replicato su più server
- ◎ Le prestazioni possono dipendere dai canali di comunicazione (ad es. congestione della rete)
- ◎ Problemi di gestione se i server sono proprietà di organizzazioni diverse

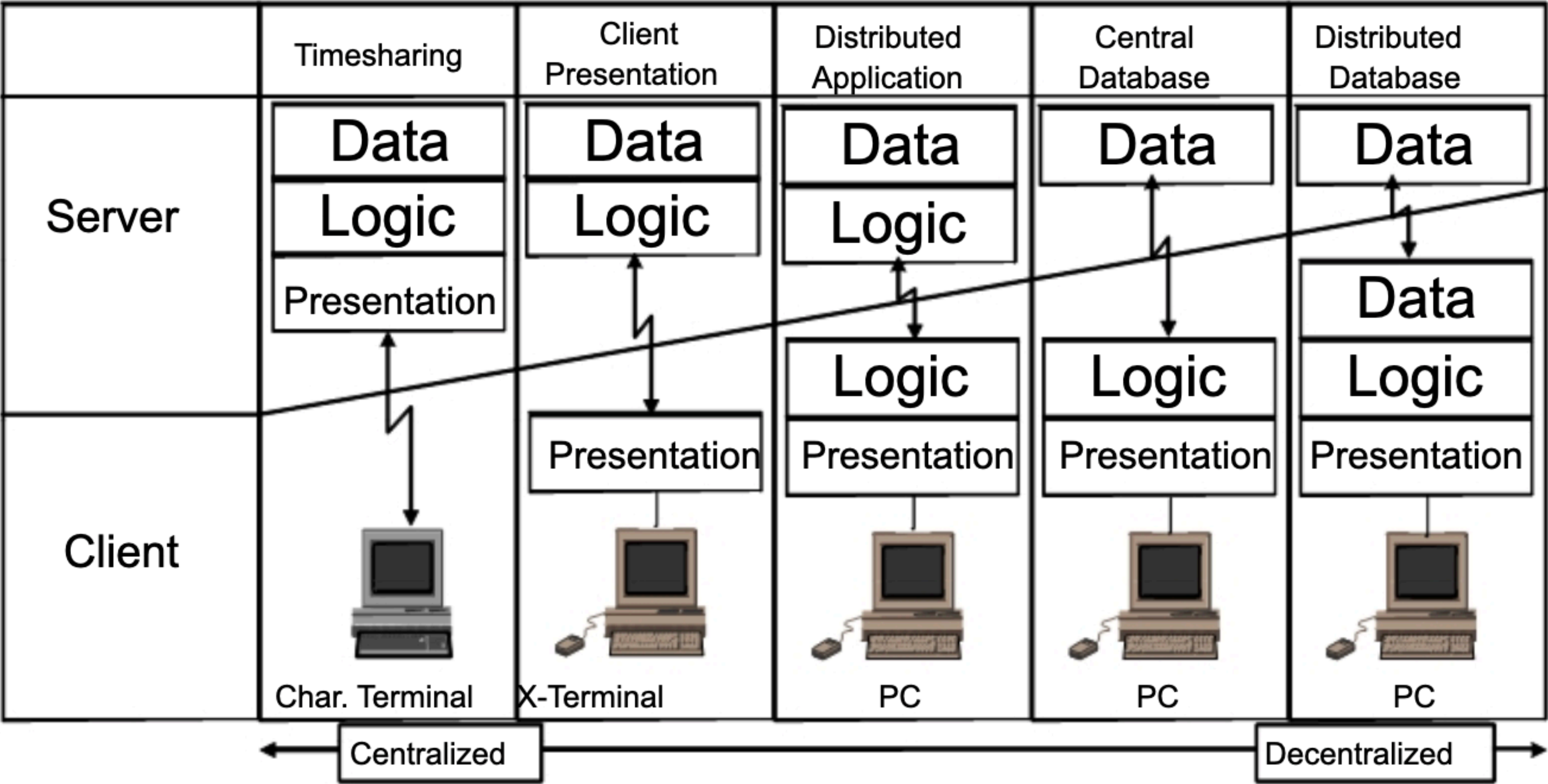


- Ogni applicazione può essere suddivisa logicamente in 3 parti:
- **Presentazione:** gestisce l'interfaccia utente (gestione eventi grafici, controlli formali sui campi in input, strumenti di supporto all'utente)
- **Logica applicativa:** implementa il cuore dell'applicazione, ovvero la logica e le regole di business che definiscono come i dati vengono elaborati per rispondere alle esigenze degli utenti
- **Gestione dei dati persistenti:** responsabile della memorizzazione e gestione dei dati a lungo termine
- La politica di allocazione di queste componenti porta alla classificazione dell'architettura: 2-tier, 3-tier, n-tier



- La distribuzione fisica e logica di queste componenti porta alla classificazione delle architetture:
 - ◎ **2-Tier:** Suddivisa in client e server. La logica applicativa può essere distribuita tra client e server.
 - ◎ **3-Tier:** I tre livelli sono separati: Presentazione sul client (es. browser), Logica applicativa su un application server e gestione dei dati su un database server. È la scelta tipica per applicazioni web scalabili
 - ◎ **n-Tier:** Estende il modello a 3 livelli con ulteriori divisioni logiche o fisiche, aggiungendo livelli come Cache Layer per ottimizzare le prestazioni e Security Layer per autenticazione e autorizzazione centralizzate. Ideale per sistemi complessi, come microservizi o applicazioni distribuite su cloud

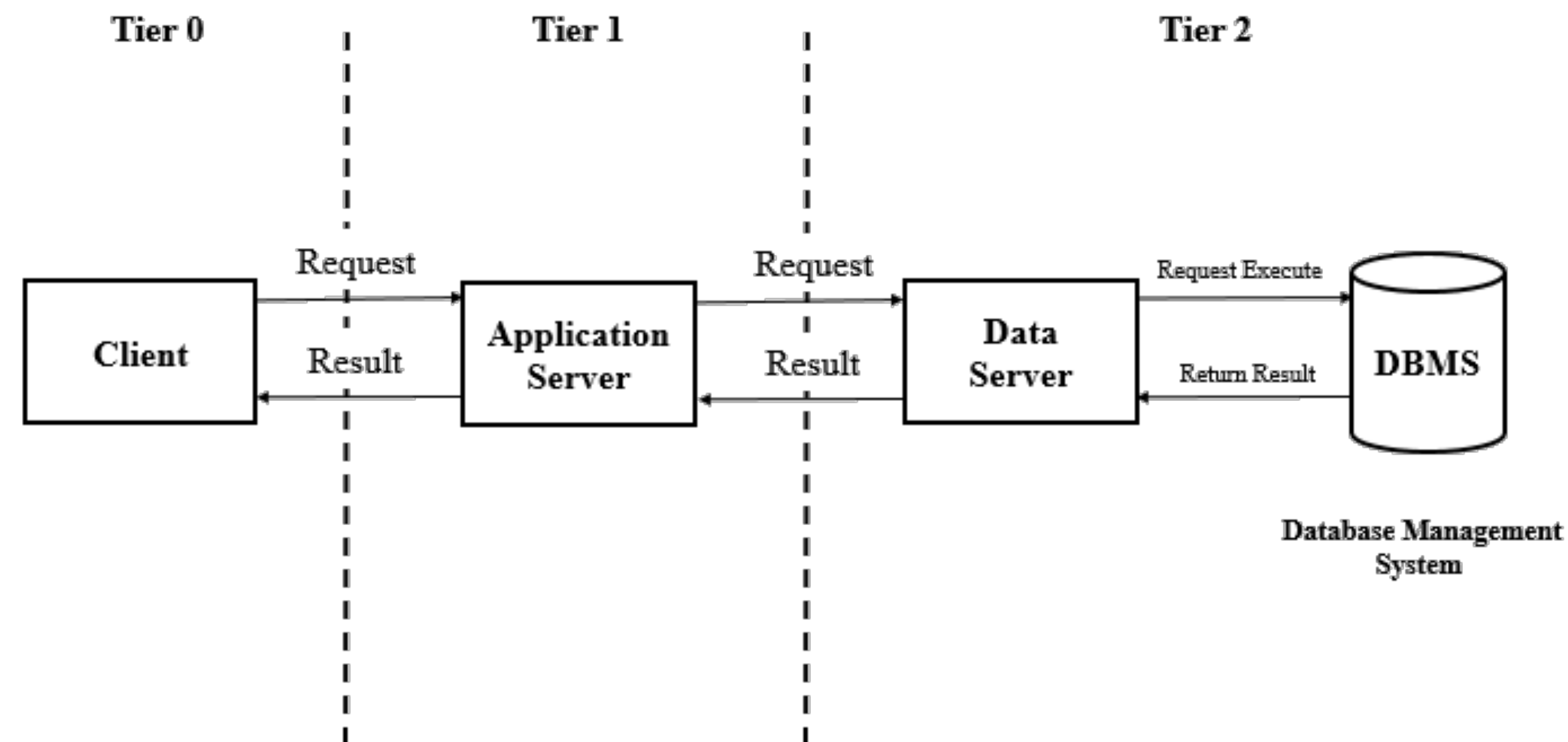
PATTERN ARCHITETTURALE CLIENT-SERVER: SOLUZIONI 2-TIER



PATTERN ARCHITETTURALE CLIENT-SERVER: SOLUZIONI 3-TIER



- Il client non comunica direttamente con il database, ma invia richieste all'application server, che le elabora e interagisce con il database.
- La separazione migliora modularità e flessibilità, distribuendo la logica su più server per gestire carichi maggiori

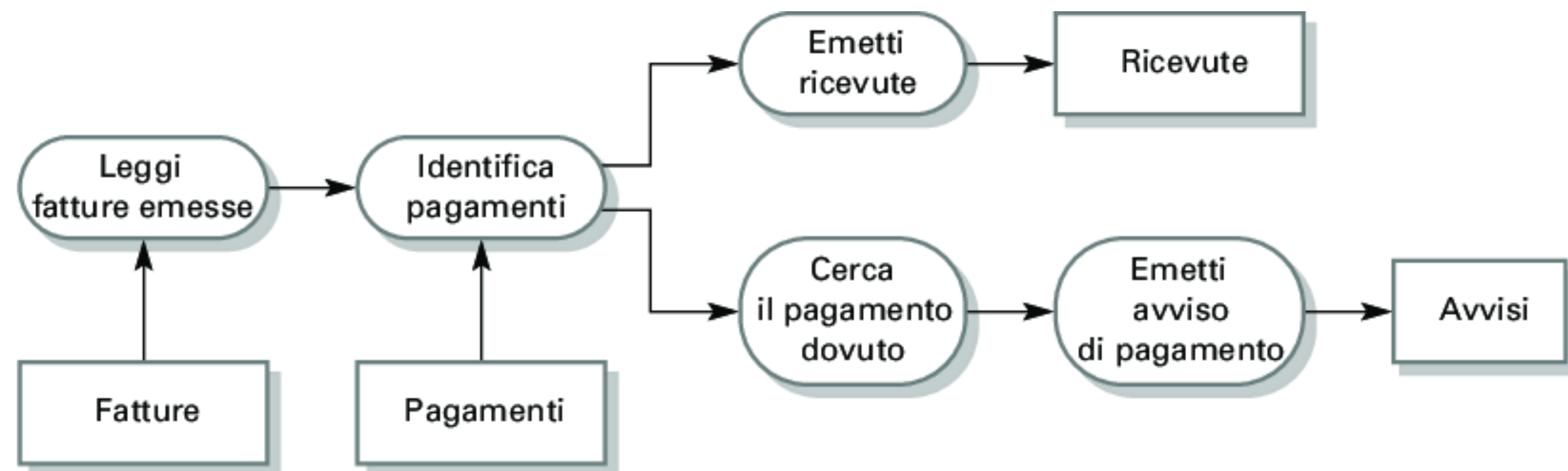


PATTERN ARCHITETTURALE PIPE-AND-FILTER



- Ciascun componente di elaborazione (filtro) é discreto e svolge una particolare trasformazione dei dati in input e produce dei dati trasformati in output
- I dati fluiscono (singolarmente o in batch) come in un tubo (pipe) attraverso una sequenza di componenti per essere elaborati
- Le trasformazioni possono essere eseguite in sequenza o in parallelo

UTILIZZATO DI SOLITO
PER L'ELABORAZIONE
DEI DATI ATTRAVERSO
SEQUENZE DI
TRASFORMAZIONI





► **Vantaggi**

- ◎ Semplice evoluzione del sistema, modificando o aggiungendo nuove trasformazioni
- ◎ Le trasformazioni possono essere riusate
- ◎ I componenti possono essere progettati separatamente

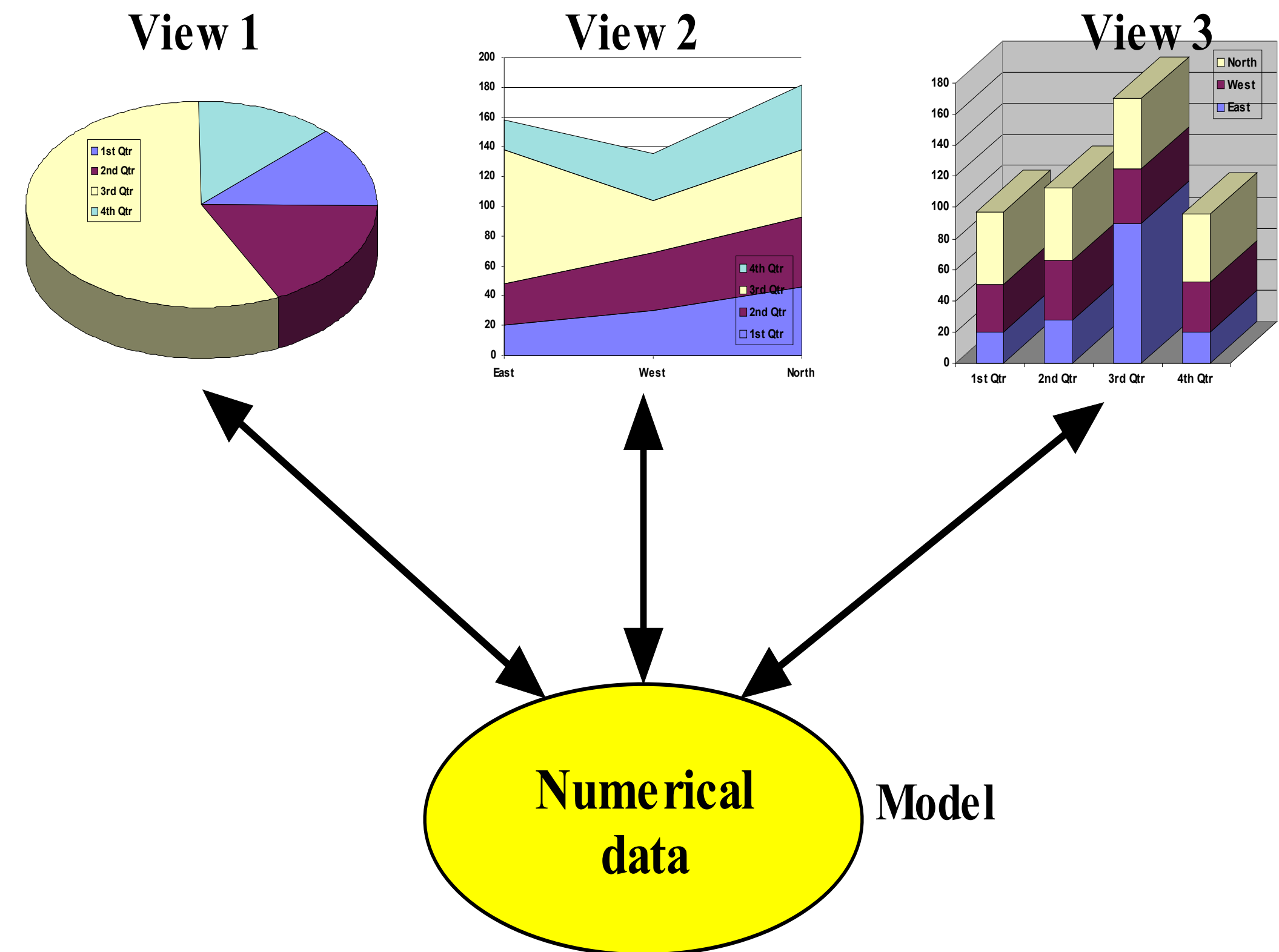
► **Svantaggi**

- ◎ Trasformazioni successive devono rispettare il formato concordato per il trasferimento dati
- ◎ Potrebbe essere impossibile riusare componenti che usano strutture dati non compatibili o introdurre overhead per la traduzione del formato
- ◎ L'interazione con gli utenti è limitata (le interfacce con gli utenti di solito non forniscono input come semplici flussi sequenziali continui)

PATTERN ARCHITETTURALE MODEL-VIEW-CONTROLLER (MVC)



- ▶ Nasce dalla necessità di visualizzare dati generici tramite interfaccia grafica (GUI), utilizzando rappresentazioni diverse dei dati stessi
- ▶ Consente la realizzazione di una architettura che permette la separazione netta dei componenti di presentazione dei dati dai componenti che gestiscono i dati stessi



PATTERN ARCHITETTURALE MODEL-VIEW-CONTROLLER (MVC)



- Le tre componenti del MVC interagiscono tra di loro
 - ⊙ Il **model** contiene le classi le cui istanze rappresentano i dati da visualizzare e manipolare. Tali classi forniscono anche i metodi per manipolare i dati
 - ⊙ La **view** contiene oggetti usati per presentare all'utente i dati contenuti nel model
 - ⊙ Il **controller** contiene gli oggetti che controlleranno e gestiranno l'interazione dell'utente sia con il livello view che il model



- Definisce le regole di business per l'interazione con i dati, esponendo alla View ed al Controller rispettivamente le funzionalità per l'accesso e l'aggiornamento
- Ha la responsabilità di notificare ai componenti della View eventuali aggiornamenti che si sono verificati in seguito a richieste del Controller, al fine di permettere alle View di presentare dati sempre aggiornati

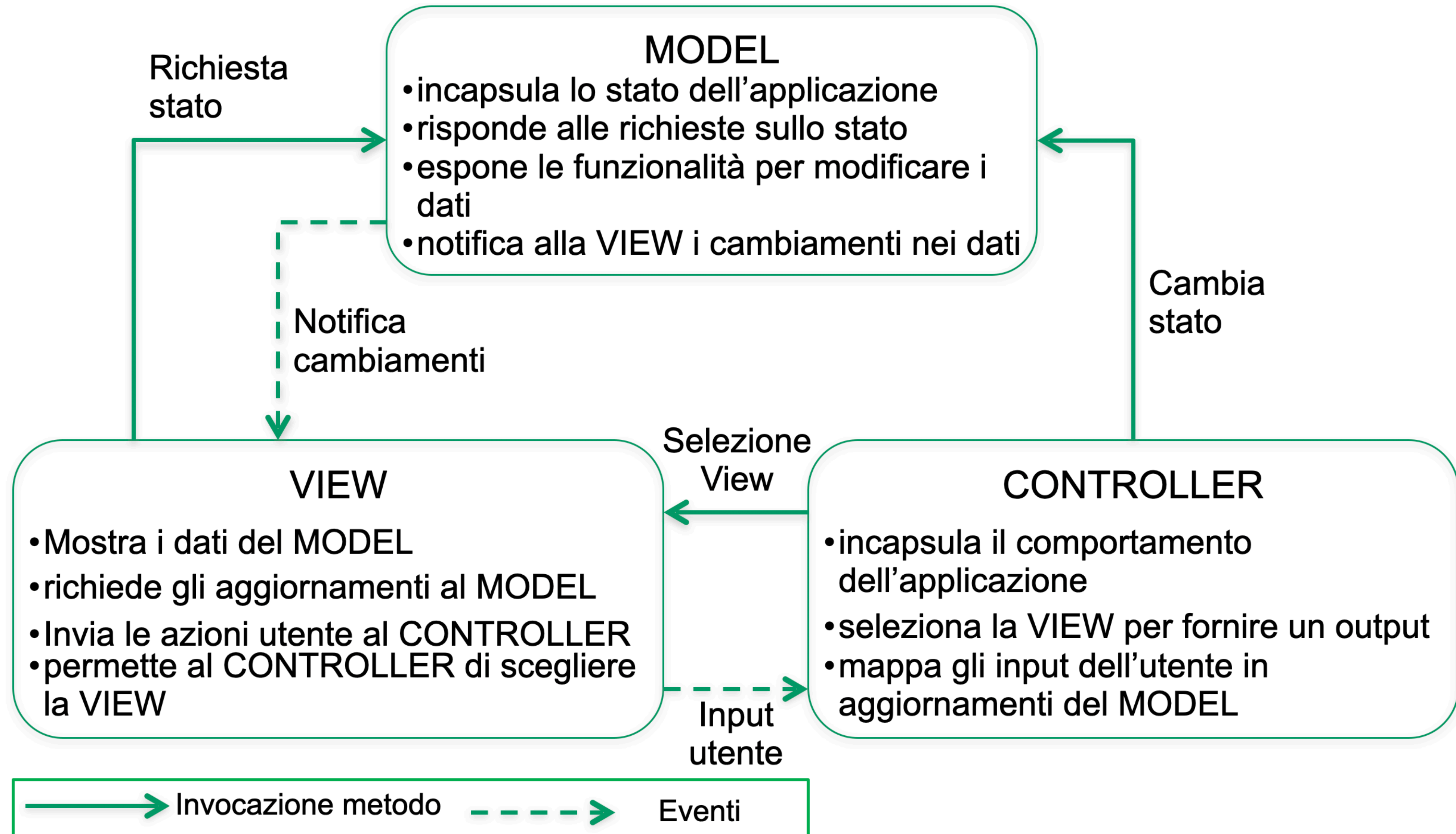


- Ha la responsabilità di trasformare le interazioni dell'utente sulla View in azioni eseguite sul Model
- Realizza la corrispondenza tra l'input dell'utente e i processi eseguiti dal Model
- Selezionando la schermate della View necessarie, il Controller implementa la logica di controllo dell'applicazione



- Gestisce la logica di presentazione dei dati
- Ogni schermata può implementare viste diverse sui dati, e modalità diverse di interazione coi dati
- Può richiedere gli aggiornamenti dei dati al Model in modo da presentare dati sempre aggiornati
- Inoltre la View delega al Controller l'esecuzione dei processi richiesti dall'utente dopo averne catturato gli input, e la scelta delle eventuali schermate da presentare

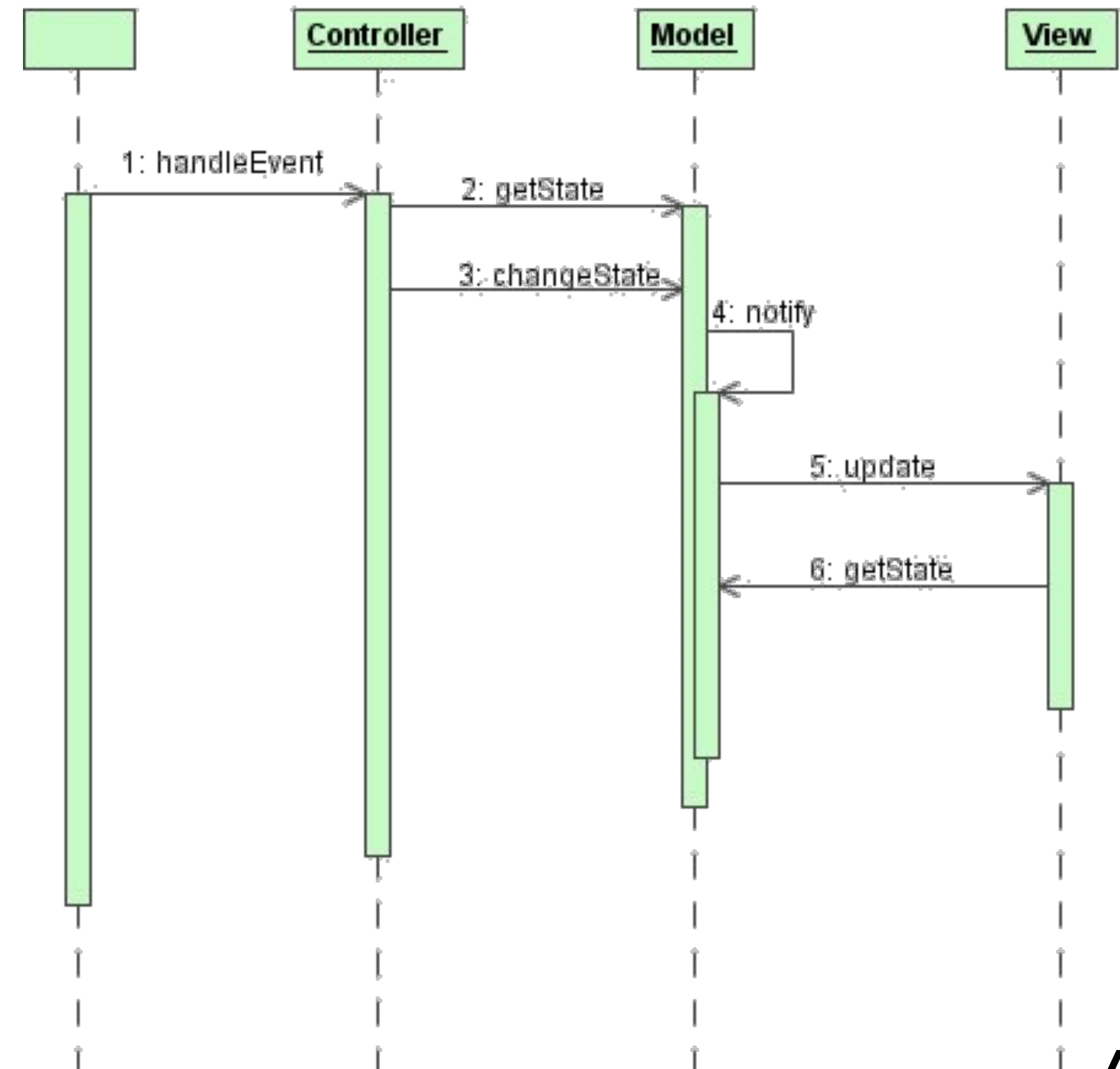
PATTERN ARCHITETTURALE MODEL-VIEW-CONTROLLER (MVC)



MVC – AGGIORNAMENTO DEI DATI



- In risposta a qualche evento il metodo `handleEvent()` viene invocato sul Controller
- Il Controller esamina lo stato di Model e invoca un suo metodo per cambiare il suo stato
- Model cambia lo stato e notifica il cambiamento di stato a tutte le View registrate
- Ogni View registrata viene notificata invocando il suo metodo `update()`. La View esamina lo stato di Model tramite il metodo `getState()`



MVC – SELEZIONE SCHERMATA



- L'utente seleziona una pagina da mostrare attraverso la View
- La View delega al Controller tramite l'invocazione del suo metodo `processSelectPage()`
- Il Controller decide e/o compone la pagina da visualizzare, e la comunica alla View invocando il suo metodo `showSelectPage()`
- La View costruisce e presenta all'utente la schermata richiesta, eventualmente controllando se ci sono state modifiche in Model invocando il metodo `getState()`

